

1. Src 분석

mock NLP workflow로써 다음의 두개를 실행하는 것이 목적이다.

1. train.py: 학습을 수행한다
2. infer.py: 학습된 weight로 inference를 수행한다 (학습이 잘 되었는지 판단한다)

소스코드는 총 6개의 파일로 구성되어 있으며 train.py를 위한 코드와 infer.py를 위한 코드로 분리할 수 있다.

train.py를 위한 코드

1. train.py: 학습을 수행한다
 - Path 구성, NLP Model 불러오기, 데이터 불러오기, trainer 호출, 도출된 weight를 저장의 순서를 수행한다
2. trainer.py: 어떻게 학습을 수행할지 그 세부적인 내용을 Trainer class로 작성한다.
 - 데이터를 불러오고, 모델의 규칙에 따라 가중치를 변경하고, 이 시간을 측정한다
3. data_handler.py: 데이터를 불러오는 역할을 수행한다
 - 문장을 word단위로 분해하고, 각각의 weight를 0.5로 초기화한다.
4. model.py: weight를 저장하고, 불러들이는 코드

infer.py를 위한 코드

1. infer.py
2. inferencer.py: 저장된 weight로 입력 문장의 score을 측정한다.
 - 문장을 word 단위로 분해하고, 해당 단어가 model.weight에 있다면 해당 weight를 score에 더한다.

전체 흐름:

data_handler.py: (저장된) 문장 단위 입력을 token stream으로 변환하고
vocabulary/count 생성

↓

trainer.py: vocabulary를 기반으로 model weight 초기화 및 mock update 수행

↓

model.py: weight 상태 관리, 저장, 불러오기

↓

입력 텍스트 파일

↓

inferencer.py: 입력 텍스트에 대한 저장된 weight를 이용해 입력 문장 score 계산

2. 개선 과정

이 과정에서는 각각의 파일에서 어느 문제점이 발생하고, 어떠한 개념을 적용하면 이것을 해결할 수 있는지 명시하고, 결과적으로 코드를 어떻게 수행하였는지 기술하였습니다.

2.1 data_handler.py의 문제 확인 및 개선

이 코드는 데이터를 로드하는 부분으로 다음의 순서로 코드가 진행된다.

1. data_path를 연다 (존재하지 않다면 FileNotFoundError)
2. 파일의 데이터를 모두 읽어온다 (Eager load problem)
3. 파일의 데이터를 word 단위로 분해한다.
4. word를 unique_word로 변환합니다.
5. self.model.top_k_words 개의 단어에 대해, 해당 단어가 몇번 등장하는지 순회합니다. (List는 이런 순회에서 비효율적)
 - 주의: 빈도 기준이 아님. 앞에서부터 순차적으로 참조하는 구조입니다.
6. 앞에서 선택된 단어들의 weight를 0.5로 초기화합니다.

코드에서 발생하는 문제점:

이 코드에서는 2가지 문제점이 발생합니다.

- B. Iterator, Generator, Lazy Evaluation 기반 개선

우선, 2번 과정에서 Eager load problem이 발생합니다. 파일의 모든 데이터를 한 번에 메모리에 올리는 구조를 취하고 있습니다.

- A. 자료구조 및 복잡도 기반 개선

두 번째 문제는, 5번에서 발생하는 list.count() 연산입니다. List에서 count는 List 전

체를 순회하면서 등장한 횟수를 세는데, 이렇게 List 전체를 순회하는 것이 비효율적입니다. 즉, $O(N * k)$ 의 asymptotic cost가 소모됩니다. if $k \approx N$, $O(N^2)$

- C: Class 기반 개선

inference.py에서 handler_data의 값을 직접적으로 참고하는 문제가 있었습니다.

- C: Class 기반 개선

sentence.strip().split()을 두 번 구현하고 있습니다. (inferencer.py에서도 사용)

이는 바람직하지 않기때문에, data_handler.py에서 sentence.strip().split()을 함수로 구현한 후 inferencer.py에서 참조하도록 수정하였습니다.

- C: Class 기반 개선

Model의 가중치를 직접 수정하는 문제가 발생하고 있습니다.

(반복적인 서술을 막기 위해) 이와 관련된 문제는 참조당하는 쪽에서 기술하겠습니다.

- D: Decorator 기반 개선

timer를 naive하게 사용하였습니다. timer 방식이 여러 코드에서 반복적으로 사용될 수 있으며 실제 동작과 혼합되어 가독성을 떨어뜨릴 수 있습니다. 공통적인 시간 측정 decorator을 정의하고 이를 통해 중복된 사용을 줄이도록 하겠습니다.

부연설명: 시간 측정은 핵심로직에서 다소 벗어난 반복적으로 사용되는 부가기능입니다. 시간 측정을 분리하면 핵심 로직과 벗어난 시간 측정 과정을 제거하고 본연의 기능에 집중할 수 있습니다.

다만 (세부적인 함수로 충분히 분리되지 않았을 경우) 함수 내부의 과정을 별도로 측정할 수 없는 문제가 있습니다. 만약 함수의 내부 과정에 대해 별도로 측정하고 싶다면, 이 과정을 별개의 함수로 분리하거나 별도의 시간 측정 과정이 필요합니다.

어떠한 개념을 적용하는가? 어떻게 수정하는가?:

generator 기법, 해시 기반 자료구조의 개념을 적용하면 된다.

generator을 사용하면 모든 데이터를 메모리에 올리지 않고, 적은 메모리 공간으로 처리할 수 있습니다.

해시 기반 자료구조를 사용하면 자료 전체를 순회하지 않고도 쉽게 확인할 수 있습니다

다. 다만, 가장 대표적인 해시 기반 자료구조인 dictionary, set에서는 같은 키를 여러번 사용할 수 없기에 단순한 변환으로는 처리가 힘듭니다.

이를 해결하기 위해서는, dict에 삽입할 때 중복 여부를 확인 후 count를 별도로 저장하거나 별도의 library에 있는 자료형을 사용해야 합니다.

이 경우 collections의 Counter 자료형을 사용하여 보다 간편하게 처리하는 방식을 선택하였습니다.

Counter를 List를 한 번 순회하여 생성함으로써, Count가 저장된 Dict를 획득할 수 있습니다.

즉, 처음 생성 비용 $O(N)$ 이후, 평균 $O(1)$ 의 비용으로 데이터에 접근 가능하게 됩니다.

개선 과정:

- B. Iterator, Generator, Lazy Evaluation 기반 개선 (Eager load problem)
- C: Class 기반 개선 (sentence.strip().split()을 두 번 구현)
(inference.py에서 handler_data의 값을 직접 참조)

수정 전 코드:

```
# BAD: Eager loading massive dataset into memory
with open(self.data_path, "r", encoding="utf-8") as f:
    self.lines = f.readlines()

for line in self.lines:
    self.all_words.extend(line.strip().split())
```

수정 후 코드 구조:

```
def split_sentence(sentence: str):
    return sentence.strip().split()
```

```

def iter_words(self):
    from inferencer import split_sentence

    with open(self.data_path, "r", encoding="utf-8") as f:
        for line in f:
            yield from split_sentence(line)

def iter_batches(self, batch_size):
    batch = []
    with open(self.data_path, "r", encoding="utf-8") as f:
        for line in f:
            batch.append(line)
            if len(batch) == batch_size:
                yield batch
                batch = []
    if batch:
        yield batch

```

설명: 이 변경의 핵심은 파일 전체를 readlines()로 한 번에 메모리에 올리지 않고, 필요한 시점에 한 줄씩 읽도록 만든 것입니다. 전체 파일을 동시에 저장할 필요가 없으며, peak memory 감소에 직접적으로 기여합니다. 또한 iter_batches()를 통해 trainer.py에서 data_handler에 직접 접근하지 않아 책임이 DataHandler로 이동하였습니다.

- D: Decorator 기반 개선 (timer을 naive하게 사용하였습니다.)

수정 전 코드:

```

start_time = time.time()
end_time = time.time()

```

수정 후 코드 구조:

```

def timed(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"[{end_time}] {func.__name__} took {end_time - start_time:.4f}s.")
        return result
    return wrapper

```

설명: 이 과정은 시간 측정이라는 부가 기능을 핵심 로직에서 분리합니다. 함수 내부에

핵심 로직과 무관한 내용을 적지 않아 가독성이 향상됩니다. 코드의 중복을 줄이고 이 방식을 중복 적용할 수 있습니다.

- A. 자료구조 및 복잡도 기반 개선 (비효율적인 list.count() 연산)

수정 전 코드:

```
# BAD: O(N^2) list counting
unique_words = list(set(self.all_words))
for word in unique_words[:self.model.top_k_words]:
    self.word_counts[word] = self.all_words.count(word)
```

수정 후 코드 구조:

```
Count_words = Counter(self.all_words)
unique_words = list(set(self.all_words))

for word in unique_words[:self.model.config.top_k_words]:
    self.word_counts[word] = Count_words[word]
```

설명: 기존 방식은 매번 list 전체를 순회하는 비효율적인 방식입니다. 수정된 코드에서는 최초에 O(N)의 비용으로 Counter 구조를 생성하고 이후 접근에서 평균적으로 O(1)의 비용으로 접근할 수 있습니다.

- C: Class 기반 개선 (Model의 가중치를 직접 수정하는 문제)

수정 전 코드:

```
for word in self.word_counts:
    self.model.weights[word] = 0.5 # initial mock weight
```

수정 후 코드 구조:

```
self.model.initialize_weights(self.word_counts)
```

고려했지만 적용하지 않은 대안:

List를 한 번 순회하면서 값을 하나씩 Dict에 추가하면서, 이미 존재할 경우 값을 1씩 증가시키는 방식도 가능하다. 다만 코드의 가독성 측면에서 더 복잡한 것 같아 Counter을 사

용하는 것을 우선하였습니다.

```
for word in self.all_words:
    if word in self.word_counts:
        self.word_counts[word] += 1
    else:
        self.word_counts[word] = 1
```

2.2 inferencer.py의 문제 확인 및 개선

해당 코드는 sentence를 입력으로 받아, 해당 sentence의 score를 계산하는 코드입니다.

1. parameter로 sentence: str을 입력받습니다.
2. sentence를 word 단위로 분해합니다.
3. self.model.weights을 확인하여 가중치가 있다면 부여합니다.
4. Score을 추력합니다. (+ 시간)

코드에서 발생하는 문제점:

- C: Class 기반 개선

model의 내부 정보를 직접적으로 가져오는 문제

(반복적인 서술을 막기 위해) 이와 관련된 문제는 참조당하는 쪽에서 기술하겠습니다.

- C: Class 기반 개선

sentence.strip().split()을 두 번 구현하고 있습니다. (data_handler.py에서도 사용)

이는 바람직하지 않고, data_handler.py에서 sentence.strip().split()을 함수로 구현하여

- D: Decorator 기반 개선

timer을 naive하게 사용하였습니다. 한 번 정의한 시간 측정 decorator을 가져와 사용함으로써 중복적인 코드 기입을 피하고 유지보수성을 향상하였습니다.

어떠한 개념을 적용하는가? 어떻게 수정하는가?:

`sentence.strip().split()`을 두 번 구현한 문제는 `sentence.strip().split()`을 별도의 함수로 분리하면 간단하게 해결할 수 있습니다.

이를 통하여 중복 코드 및 책임 문제를 더 명확히하여 코드의 유지보수성 향상시킵니다.

`timer` 역시 `Decorator`을 사용하여 간단하게 해결할 수 있습니다.

`timer`은 반복적으로 사용되고, `Decorator`을 사용하여 중복코드 제거하고, 유지보수성을 향상시킵니다.

개선 과정:

- C: Class 기반 개선 (model의 내부 정보를 직접가져오는 문제)

수정 전 코드:

```
for word in words:
    # Reaches directly into the model's dictionary
    if word in self.model.weights:
        score += self.model.weights[word]
```

수정 후 코드 구조:

```
for word in words:
    score += self.model.get_weight(word)
```

설명: 기존에는 Inference에서 model의 정보를 직접적으로 참조하고 있었습니다. 이 구조에서는 model의 내부 구조가 변경될 경우 model을 참조하는 다른 class/function을 찾아 수정해야 합니다.

수정된 버전에서는 책임을 명확히 분리하여 model의 수정은 model에서 끝나도록 수정하였습니다.

- C: Class 기반 개선 (`sentence.strip().split()`을 두 번 구현)

수정 전 코드:


```
words = sentence.strip().split()
```

수정 후 코드 구조:

```
words = split_sentence(sentence)
```

설명: 이 과정은 반복적으로 사용되는 과정을 하나의 함수로 정의한 것입니다.

- D: Decorator 기반 개선(timer를 naive하게 사용)

수정 전 코드:

```
start_time = time.time()
```

```
end_time = time.time()
```

수정 후 코드 구조:

```
@timed  
def infer(self, sentence: str) -> float:
```

설명: 이 과정은 시간 측정이라는 부가 기능을 핵심 로직에서 분리합니다. 함수 내부에 핵심 로직과 무관한 내용을 적지 않아 가독성이 향상됩니다. 코드의 중복을 줄이고 이 방식을 중복 적용할 수 있습니다.

2.3 model.py의 문제 확인 및 개선

model.py에서는 가중치를 저장하고 불러오는 역할을 수행하고 있습니다.

코드에서 발생하는 문제점:

C: Class 기반 개선

우선, learning_rate, epochs, batch_size를 다루는 데 있어 특별한 조치가 취해져있지 않습니다.

이 값을 변경하기 위해서는 Model에서 직접적으로 수정해야 합니다.

또한 각 설정값에 대한 검증 로직이 없기 때문에, 잘못된 값이 들어와도 이를 바로잡지 못합니다.

C: Class 기반 개선

두 번째로, Model 내부의 값을 다른 Class / function에서 직접 수정하는 문제가 있었습니다.

캡슐화를 위반하여 Model와 다른 Class 사이의 결합도를 높입니다.

Model의 내부에 변동이 있을 경우 Model을 직접적으로 참조하는 다른 코드를 찾아서 수정해야 합니다.

이는 유지보수성에 매우 불리하며, 모든 책임을 Model Class 내부로 모을 필요가 있습니다.

어떠한 개념을 적용하는가? 어떻게 수정하는가?:

잘못된 초기값의 문제를 해결하기 위해 값을 검증하는 과정이 추가되어야 합니다.

추가적으로, 이 코드에서는 설정의 유연한 변경이 필요한 코드가 아니라 판단하고 @dataclass(frozen=True)를 사용하고 __post_init__으로 검증하는 방식을 사용하였습니다.

이는, hyper parameter에 대한 가독성을 향상하는 한편, 잘못된 값에 대한 검증 역시 수행합니다.

- 단점: @data_class의 사용은 hyper parameter를 자주 수정하지 않는다는 가정 하에 작성한 것으로, 만약 실험 도중 유동적인 hyper parameter의 수정이 필요한 경우 올바른 접근이 아닐 수 있다.

Model의 내부 값을 다른 함수에서 직접 수정하는 문제의 경우, 함수를 통하여 접근하도록 수정하였습니다.

Model의 내부 문제는 Model에게 책임을 위임하여, Model 내부의 수정이 다른 코드에 영향을 끼치지 않게 되었습니다.

- 단점: 내부 상태로의 직접적인 접근을 줄여 책임을 분리하여 유지보수성을 향상하지만, 많은 함수가 생성되어 구조가 복잡해보일 수 있다.

개선 과정:

C: Class 기반 개선 (초기 값에 대해 적절한 체크가 없음)

수정 전 코드:

```
def __init__(self):
    # Configurations hardcoded
    self.learning_rate = 0.001
    self.epochs = 3
    self.batch_size = 1000
    self.top_k_words = 20

    # Runtime State
    self.weights = {}
```

수정 후 코드 구조:

```
@dataclass(frozen=True)
class ModelConfig:
    learning_rate: float = 0.001
    epochs: int = 3
    batch_size: int = 1000
    top_k_words: int = 20

    def __post_init__(self):
        if self.learning_rate <= 0:
            raise ValueError("learning_rate must be positive")
        if self.epochs < 0:
            raise ValueError("epochs must be positive")
        if self.batch_size < 0:
            raise ValueError("batch_size must be positive")
        if self.top_k_words < 0:
            raise ValueError("top_k_words must be positive")
```

설명: 기존에는 model의 초기 정보가 model에 하드코딩되어 있었습니다. 이 과정을 config로 분리하여 가독성을 향상하였고, __post_init__ 사용하여 초기 값을 검증하는 과정을 추가하였습니다.

C: Class 기반 개선 (Model 내부의 값을 다른 함수/클래스에서 직접 수정)

수정 전 코드:

- 별도의 과정 없이 내부값을 직접 수정

수정 후 코드 구조:

```
@property
def weights(self):
    return dict(self._weights)

def initialize_weights(self, words, initial_weight=0.5):
    for word in words:
        self._weights[word] = initial_weight

def update_all_weights(self, delta):
    for word in self._weights:
        self._weights[word] += delta

def get_weight(self, word, default=0.0):
    return self._weights.get(word, default)
```

설명: 다른 함수/클래스에서 model의 정보를 직접적으로 참조하고 있었습니다. 이 구조에서는 model의 내부 구조가 변경될 경우 model을 참조하는 다른 class/function을 찾아 수정해야 합니다.

수정된 버전에서는 책임을 명확히 분리하였습니다.

2.4 trainer.py의 문제 확인 및 개선

데이터를 불러오고, 규칙에 따라 가중치를 수정하는 역할을 수행합니다.

코드에서 발생하는 문제점:

이전에 이미 지적되어 수정된 문제:

1. time을 코드 내부에서 직접 측정하고 있었습니다. 이전에 지적한대로 Decorator을 사용합니다.
2. Model의 내부를 직접적으로 수정하고 있습니다. Model의 function을 이용함으로써 책임을 분리합니다.
- + . Model의 내부 정보를 config를 이용하여 깔끔하게 참조합니다.

새롭게 발생한 문제:

C: Class 기반 개선

DataHandler class에 직접 접근하는 문제가 발생하고 있습니다. 앞서 언급하였듯이, 이렇게 캡슐화를 위반하면 유지보수성을 해치게 됩니다.

-> DataHandler class에서 함수를 추가하여 책임을 DataHandler class에 위임합니다.

코드에서 발생하는 문제점: 어떠한 개념을 적용하는가? 어떻게 수정하는가?:

참조되는 쪽에서 자세하게 기술하고 있기에, 이 부분에서는 자세한 설명을 생략하겠습니다. (반복적인 서술이 되기에)

개선 과정:

- D: Decorator 기반 개선(timer를 naive하게 사용)

수정 전 코드:

```
start_time = time.time()
end_time = time.time()
```

수정 후 코드 구조:

```
@timed
def infer(self, sentence: str) -> float:
```

설명: 이 과정은 시간 측정이라는 부가 기능을 핵심 로직에서 분리합니다. 함수 내부에 핵심 로직과 무관한 내용을 적지 않아 가독성이 향상됩니다. 코드의 중복을 줄이고 이 방식을 중복 적용할 수 있습니다.

- C: Class 기반 개선 (Model의 내부를 직접적으로 수정)

수정 전 코드:

```
for word in self.model.weights:
    self.model.weights[word] += 0.001
```

수정 후 코드 구조:

```
self.model.update_all_weights(0.001)
```

설명: 기존에는 trainer.py에서 model의 정보를 직접적으로 참조하고 있었습니다. 이 구조에서는 model의 내부 구조가 변경될 경우 model을 참조하는 다른 class/function을 찾아 수정해야 합니다.

수정된 버전에서는 책임을 명확히 분리하여 model의 수정은 model에서 끝나도록 수정하였습니다.

- C: Class 기반 개선 (DataHandler class에 직접 접근)

수정 전 코드:

```
for i in range(0, lines_count, batch_size):
    batch = self.data_handler.lines[i:i+batch_size]

    # Mock forward pass and update
    loss_step = sum(len(line) for line in batch) * lr
    total_loss += loss_step
```

수정 후 코드 구조:

```
for epoch in range(epochs):
    total_loss = 0.0

    for batch in self.data_handler.iter_batches(batch_size):
```

설명: 기존에는 trainer.py에서 DataHandler의 정보를 직접적으로 참조하고 있었습니다. 이 구조에서는 model의 내부 구조가 변경될 경우 model을 참조하는 다른 class/function을 찾아 수정해야 합니다.

수정된 버전에서는 책임을 명확히 분리하여 DataHandler의 수정은 DataHandler에서 끝나도록 수정하였습니다.

3. 개선 정리

A. 자료구조 및 복잡도 기반 개선

적용 위치	기존 문제	수정 방향
data_handler.py	단어의 수를 계산할 때, List의 순회를 통해 이루어지며 매우 비효율적으로 이루어졌습니다. $O(N * k)$	해시 기반 자료구조인 collections.Counter을 사용하여 단어 빈도를 한 번에 알 수 있도록 수정하였습니다.

B. Iterator, Generator, Lazy Evaluation 기반 개선

적용위치	기존 문제	수정 방향
data_handler.py	f.readlines()로 한 번에 읽어와 저장하는 Eager load 문제가 발생하고 있었습니다.	generator iter_words()를 추가하여 한 줄씩 읽고, 각 줄에서 분해한 단어를 return 하는 방식으로 수정하였습니다.
trainer.py (data_handler.py)	data_handler.lines에 직접 접근하는 문제가 있었습니다.	iter_batches()를 추가하여, 종속성을 제거함과 동시에 generator을 이용하여 한 번에 한 줄씩 읽도록 수정하였습니다.

C. Class 기반 구조 개선 및 단일책임원칙 적용

적용위치	기존 문제	수정방향
model.py	Model 내부의 learning_rate, epoches 등이 별도의 검증 과정 없이 하드코딩 되어있었습니다.	(반복적인 유연성이 필요하지 않은 것 같아) @dataclass(frozen=true), config를 사용하여 가독성을 향상하고, __post_init__을 이용해 값을 검증하는 과정을 추가하였습니다.
model.py	다른 클래스나 함수에서 model의 weight를 직접 읽거나 수정하는 문제가 발생하고 있었습니다.	내부 상태를 _weights로 변경하였으며, 값을 직접적으로 변경하는 것이 아닌 update_all_weight()등으로 수정/접근하게 수정하였습니다.
trainer.py	data_handler.lines에 직접 접근하는 문제가 있었습니다.	iter_batches()를 추가하여 해결하였습니다.

inferencer.py/ data_handler.py	sentence.strip().split()를 두 곳에서 중복되게 사용하고 있는 문 제가 있었습니다.	data_handler.py에서 sentence.strip().split()를 함수로 정의하고 inferencer.py가 이를 참조하여 사용하는 방식으로 변경 하였습니다.
-----------------------------------	--	---

D. Decorator 기반 개선

적용 위치	기존 문제	수정 방향
trainer.py	기존에 함수들이 각각 시간을 측정하고 있었습니다.	timer decorator을 추가하여 이를 다른 곳에서 불러오는 것으 로 유지보수성을 향상하였습니 다.

4.0 결과 정리

4.1 github 구조

```

SampleProject/
├── src/
│   ├── data/
│   │   ├── dataset_10mb.txt
│   │   ├── dataset_40mb.txt
│   │   └── dataset_160mb.txt
│   ├── before/
│   │   ├── train.py
│   │   └── infer.py
│   └── after/
│       ├── train.py
│       └── infer.py
├── benchmark/
│   └── run_benchmark.py
└── results/

```



```
|—— benchmark_results.csv
|—— graph/
|   |—— benchmark_train_average_time.png
|   |—— benchmark_infer_average_time.png
```

4.2 최적화 결과

우선, benchmark/run_benchmark.py를 어떻게 수행하였는지 설명하겠습니다.

개선 전후를 보다 정량적으로 비교하기 위하여 우선, 데이터의 크기를 10MB, 40MB, 160MB 크기의 3가지 입력 데이터를 사용하였습니다.

또한 각각의 데이터에 대해 Before, After에서 모든 연산을 10번씩 수행하였습니다.

이러한 환경에서 평균 실행 시간, 실행 시간의 표준편차, 평균 메모리 사용량, peak memory를 측정하였습니다. 이 결과를 전체적으로 results/benchmark_results.csv에 저장하였습니다.

또한 보다 나은 가시성을 제공하기 위하여, 실행시간을 10MB / 40MB / 160MB의 환경에서 Before / After의 실험 결과를 각각 results/graph/benchmark_train_*.time.png에 저장하였습니다

4.2에서는 결과를 간략히 나열하고 이에 대한 분석은 4.3에서 수행하겠습니다.

우선 속도 측면에서 설명하겠습니다.

train의 경우 전체적인 환경에서 속도가 감소하였습니다.

infer의 경우는 특별한 개선을 확인할 수 없었고, 데이터의 양에 따른 차이도 확인할 수 없었습니다.

메모리 측면에서 설명하겠습니다.

train의 경우 전체적인 환경에서 평균 메모리, peak 메모리가 감소하였습니다.

infer의 경우 반대로, 전체적인 환경에서 평균 메모리가 증가하는 경향을 보였습니다. peak의 경우 경향성을 확인할 수 없었습니다.

코드의 구조 및 유지모수성은 4.3에서 설명합니다.

4.3 결과 해석 및 한계

여기서는 4.2에서 정리한 benchmark 결과가 발생한 이유를 실행 시간, 메모리 사용량, 코드 구조 측면에서 설명합니다.

먼저 실행 시간 측면에서 보면, `train.py`에서 개선 효과는 자료구조적 개선에서 발생합니다. 기존 코드에서는 단어의 등장 횟수를 계산할 때, `List.count()`를 반복적으로 호출합니다. 이 과정에서 $O(N * k)$ 의 연산량이 발생하는 비효율적인 부분이었습니다.

이 부분을 개선하여, `collections.Counter`를 사용하여 최초의 한 번 $O(N)$ 의 비용으로 Count를 생성하고 이후에는 $O(1)$ 으로 접근 가능하게 한다. 이에 따라 조회가 보다 효율적으로 이루어지게 하였습니다.

이 부분의 개선으로 인하여 train.py의 실행 시간이 전반적으로 감소하였습니다.

반면, infer.py에서는 뚜렷한 개선이 관찰되지 않습니다. 이는 inference 과정에서 학습 데이터를 처리하지 않고 이에 따라 비효율적인 구조가 본래 코드에서 관찰되지 않습니다. infer.py는 저장된 weight파일을 불러와 score을 계산합니다. 또한 이 과정은 input data의 크기와 영향을 받지 않습니다. weight의 크기는 별도의 파라미터로써 input size와 무관하기 때문입니다.

메모리 사용 측면에서는 train.py에서 평균 메모리와 peak memory가 감소하였습니다. 기존 코드에서는 파일의 모든 데이터를 한 번에 메모리에 올리는 구조를 취하였습니다. 수정 후에는 generator를 사용하여 파일을 한 줄씩 읽고 필요한 단위로 처리하도록 변경하였습니다. 이러한 관찰은 데이터의 크기가 증가할때 보다 뚜렷하게 관찰됩니다.

반면, infer.py에서는 오히려 평균 메모리 사용량이 증가한 경향이 보입니다. 이는 generator, decorator 등에서 발생하는 오버헤드에 의해 발생합니다. 이러한 방식은 실행 시간을 줄이는 방식이라 하기 힘듭니다. generator의 예시로 메모리 사용량을 줄이는 데 유리하지만, 값을 필요할 때마다 생성하고 함수 호출을 거쳐 전달하기 때문에 약간의 추가 오버헤드가 발생할 수 있습니다. infer.py에서는 짧은 양만을 처리하기에 전체 시간이 짧고 이러한 경향이 관측될 수 있습니다.

코드 구조 측면의 개선은 주로 성능보다는 유지보수성을 위한 것입니다.

예를 들어 mode에서 learning_weight를 실수로 음수로 설정하더라도 기존의 코드에서는 이를 확인할 수 있는 방법이 없었습니다. 하지만 개선된 코드에서는 이런 기본적인 실수를 점검하도록 수정되었습니다.

또한 model의 내부에서 직접 수정되는 기존의 방식에서, model에 함수로 접근하여 수정하도록 수정하여 책임을 명확히 분리하였습니다. 이제 model의 내부 구조가 변경되더라도, model 내부의 수정에서 그치고 이것이 사용된 여러 코드에서 수정할 필요가 없습니다. 그리고 코드의 확장성 역시 확장되었습니다.